

STA5012 Final Group Project

Attention Linearization and Feature Maps

Due: May 10, 2026

Group size: 4 students

Suggested time budget: about one month

1 Motivation

The self-attention mechanism is one of the key ingredients in modern transformer models. For a sequence of length n , exact softmax attention computes all pairwise query-key interactions, so its time complexity is quadratic in n . In standard implementations, memory usage is also quadratic in n because one typically forms or stores an $n \times n$ attention matrix. This becomes a major bottleneck for long contexts.

A main idea behind *attention linearization* is to approximate the softmax attention kernel by a finite-dimensional feature map. If this approximation is accurate, then both computation and memory with respect to sequence length can be reduced from quadratic to linear (for fixed feature dimension). This is why finding a good feature map is both mathematically interesting and practically useful.

2 Background

For one causal attention head, let the query, key, value vectors be $q_i, k_i \in \mathbb{R}^d$ and $v_i \in \mathbb{R}^{d_v}$. At the i -th location, standard causal softmax attention calculates

$$o_i = \frac{\sum_{j \leq i} \exp(q_i^\top k_j / \sqrt{d}) v_j}{\sum_{j \leq i} \exp(q_i^\top k_j / \sqrt{d})}.$$

The quadratic dependence on sequence length comes from filling up the $i \times i$ attention scores. Each element given by a normalized version of

$$K(q, k) = \exp(q^\top k / \sqrt{d}).$$

$K(q, k)$ can be viewed as a kernel function measuring the similarity between q and k . The basic attention linearization idea is to approximate this kernel by an inner product

$$K(q, k) \approx \phi(q)^\top \phi(k),$$

where $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^m$ is a feature map of dimension m . If this approximation is good, then

$$\hat{o}_i = \frac{\phi(q_i)^\top \left(\sum_{j \leq i} \phi(k_j) v_j^\top \right)}{\phi(q_i)^\top \left(\sum_{j \leq i} \phi(k_j) \right)}$$

can be computed without explicitly forming the full attention matrix. For fixed m , this leads to linear dependence on sequence length.

3 Preliminary knowledge

A standard way to approximate kernels is through *random feature maps* [2]: instead of evaluating a kernel exactly, one maps the input into a finite-dimensional feature space and approximates the kernel by an inner product there.

One important baseline is **Performer** [1]. Performer follows this random-feature philosophy and uses a softmax-specific positive random feature construction (FAVOR+) to approximate softmax attention efficiently. You may use an existing implementation or re-implement a simplified version.

You should compare Performer with at least one additional method. This additional method may be either:

- a simple feature map that you design yourself; or
- an existing method from the literature.

The goal is not to invent a completely new architecture. The goal is to find a method that improves on Performer under the same evaluation metric.

In the real-data part of the project, you will work with a **pretrained GPT-2 small** model from Hugging Face. In this project, GPT-2 remains *frozen*: you are not asked to convert the whole model, retrain it, or fine-tune it. The goal is only to extract real query/key/value vectors from a pretrained transformer and evaluate feature-map approximations on them.

As optional background, you may also look at recent work such as Hedgehog [3], which argues that real pretrained transformers are difficult for linear attention partly because softmax attention often produces low-entropy (“spiky”) attention patterns and has strong monotonic dependence on the query-key dot product. You do not need to reproduce Hedgehog; it is only a useful reference point for interpreting why simple feature maps may work in toy settings but fail on real language-model activations.

4 Project goal

The goal of this project is to understand how well different feature maps approximate softmax attention in two settings:

- a controlled Gaussian toy setting;
- a frozen pretrained GPT-2 model on real text data.

The main point is to use the *same evaluation metric* in both settings, so that methods can be compared consistently. As the feature dimension m increases, a good method should show steadily improving approximation quality under that same metric.

5 Main evaluation metric

Let

$$K(q, k) = \exp(q^\top k / \sqrt{d}), \quad \hat{K}_\phi(q, k) = \phi(q)^\top \phi(k).$$

The main metric in this project is the *relative kernel approximation error*

$$\text{RelErr}_{\text{ker}}(\phi) = \frac{\mathbb{E}[(K(q, k) - \hat{K}_\phi(q, k))^2]}{\mathbb{E}[K(q, k)^2]}.$$

In the Gaussian setting, the expectation is with respect to the Gaussian sampling scheme. In the GPT-2 setting, it is replaced by the empirical average over extracted (q, k) pairs.

This is the required metric for all groups. Your main plots should show this error as a function of feature dimension m .

You are also encouraged to report at least one secondary metric that is closer to actual attention behavior, such as an attention-output error. For example, if A and $\hat{A}$ denote the exact and approximate causal attention matrices, you may report

$$\text{RelErr}_{\text{out}} = \frac{\|AV - \hat{A}V\|_F}{\|AV\|_F}.$$

This secondary metric is recommended but not required.

6 Tasks

Part I: Gaussian toy setting

Assume throughout that

$$q, k \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, I_d).$$

- G1.** Implement Performer’s feature map with varying feature dimension m .
- G2.** For several feature dimensions m , estimate $\text{RelErr}_{\text{ker}}(\phi)$ and plot it against m .
- G3.** Verify numerically that the approximation improves as m increases. Compare the different methods under the same metric. Does the error appear to decay toward zero as m grows?
- G4.** Treat Performer as a baseline, propose an improved feature map and verify its performance in the same benchmark.
- G5.** Discuss what should the optimal method look like in the Gaussian toy setting? What may be the best-possible performance achievable for a fixed m .

Part II: Frozen GPT-2 on real data

Use the pretrained GPT-2 small model from Hugging Face (<https://huggingface.co/openai-community/gpt2>). Choose any real data to your liking and extract query/key/value vectors from several layers (e.g., layer 1 and 7) and heads. The model should remain frozen throughout the project.

- R1.** Evaluate the same feature maps as in Part I on real GPT-2 activations.
- R2.** Use the same main metric $\text{RelErr}_{\text{ker}}(\phi)$ and plot it against feature dimension m .
- R3.** Compare the GPT-2 results with the Gaussian results. Identify where the approximation is easy and where it is difficult (for example across layers or heads).
- R4.** Inspect the coordinates of the extracted queries and keys. In particular, plot the dimension-wise means for several heads. Do the vectors look approximately centered Gaussian? If not, what kinds of deviations do you observe?
- R5.** Propose one modification or alternative feature map motivated by your observations, and test whether it gives a consistent improvement over Performer under the same metric. This feature map can be the same as in Part I as long as it is better than the Performer baseline.
- R6.** Discuss what should the optimal method look like in the real-data setting? What may be the best-possible performance achievable for a fixed m .

7 What to focus on

Before exploring further extensions, make sure the following points are clearly addressed:

- What is the approximation goal?
- How is approximation quality measured?
- Does performance improve consistently as feature dimension increases?
- Does a method that works well in the Gaussian setting still work well on real GPT-2 activations?
- If real GPT-2 activations behave very differently from Gaussian vectors, how should that affect the feature-map design?

8 Deliverables

Each group should submit the following **via email to the instructor and TA** before the deadline:

- A short report with clear figures and tables; in each setting, a method that is performing better than the Performer baseline.
- Reproducible code.

A strong report should be clear, focused, and comparative. In particular, it should include:

- the methods being compared;
- the evaluation metric(s);
- error-versus-feature-dimension plots in both settings;
- a short discussion of why the real GPT-2 case differs from the Gaussian toy case;
- a clear explanation of the idea behind the better-performing method you tested.

In the last lecture (on May 7), each group should prepare a 15-min presentation.

References

- [1] K. Choromanski, V. Likhoshesterov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser, et al. Rethinking attention with performers. *arXiv preprint arXiv:2009.14794*, 2020.
- [2] A. Rahimi and B. Recht. Random features for large-scale kernel machines. *Advances in neural information processing systems*, 20, 2007.
- [3] M. Zhang, K. Bhatia, H. Kumbong, and C. Ré. The hedgehog & the porcupine: Expressive linear attentions with softmax mimicry. *arXiv preprint arXiv:2402.04347*, 2024.